TEMA 3 · FIXTAG

# Persistência local e funcionamento offline

SharedPreferences, DataStore, ficheiros internos, SQLite e Room, aplicados a uma app de assistência técnica que continua a funcionar sem rede.

**Hugo Carvalho**

27 de outubro de 2026

ENQUADRAMENTO

# Porquê funcionar sem rede

O FixTag regista avarias em máquinas industriais — muitas vezes em pavilhões e caves onde a rede falha.

Se a app depender do servidor, a avaria fica por registar. Por isso, tudo é gravado primeiro no dispositivo e enviado quando houver rede.

GUARDADO NO DISPOSITIVO

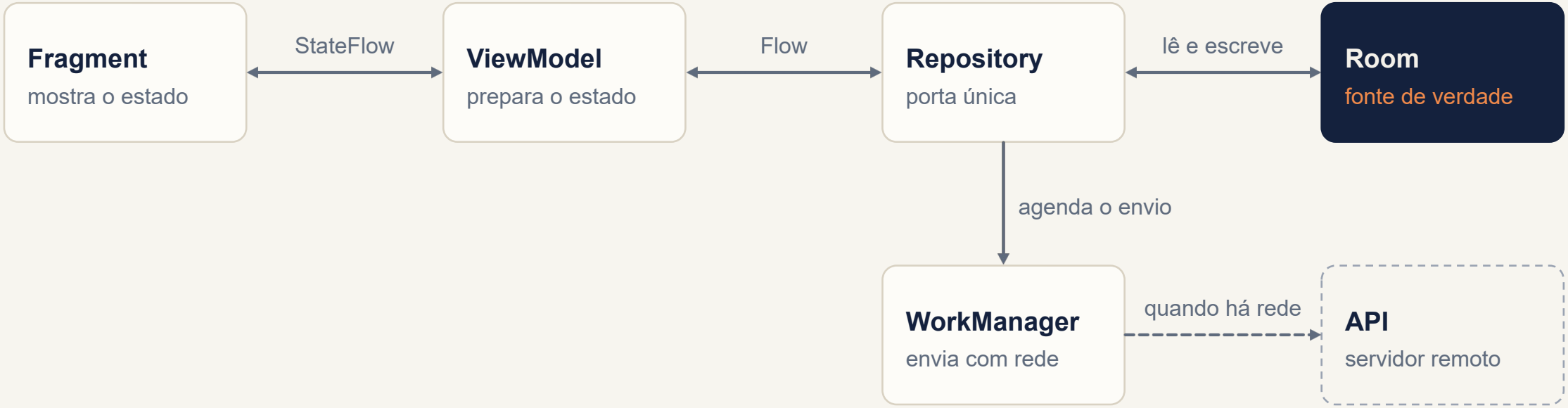
**Sessão (perfil escolhido)**
DataStore

**Organizações, máquinas, ocorrências**
Room (SQLite)

**Fotos e gravações de áudio**
Ficheiros internos

CONCEITOS

# Offline-first




A UI nunca fala com a rede: lê e escreve na Room, e a sincronização acontece em segundo plano.

Na variante demo, o WorkManager e a API estão desligados (BuildConfig.OFFLINE_DEMO).

CONCEITOS

# Opções de persistência

| Tecnologia | Para quê | API | Diretório | No FixTag |
|---|---|---|---|---|
| SharedPreferences | pares chave-valor | síncrona, ficheiro XML | shared_prefs/ | não usado |
| DataStore | chave-valor (Preferences) ou objetos tipados (Proto) | coroutines e Flow | files/datastore/ | sessão |
| Ficheiros internos | fotos, áudio, documentos | java.io.File | files/, cache/ | anexos |
| SQLite | dados relacionais | SQL e Cursor | databases/ | por baixo da Room |
| Room | dados relacionais | entidades, DAO, Flow | databases/ | máquinas, ocorrências |

Diretórios dentro de /data/data/<pacote>/ — privados da app e apagados quando é desinstalada.

DATASTORE

# SharedPreferences e DataStore

### SharedPreferences

- leitura síncrona, na main thread
- apply() pode bloquear a UI (ANR)
- sem transações nem erros visíveis

### DataStore

- coroutines e Flow, fora da main thread
- edit { } é uma escrita atómica
- os erros chegam como IOException
- Preferences (chave-valor) ou Proto (tipado)

```
// data/Session.kt — um só DataStore por ficheiro
val Context.sessionStore: DataStore<Preferences>
        by preferencesDataStore(name = "session")

// data/Repository.kt
private val currentUserId =
        stringPreferencesKey("current_user_id")

// escolher um perfil na demo = gravar o id
session.edit { it[currentUserId] = userId.toString() }

// ao abrir a app: há uma sessão guardada?
val userId = session.data.first()[currentUserId]
        ?.let(Uuid::parse)
```

Fonte: android/app/src/main/java/com/example/fixtag/data/Session.kt e Repository.kt

DATASTORE · JAVA

# SharedPreferences e DataStore em Java

```java
// SharedPreferences: API Java nativa
SharedPreferences prefs = context
    .getSharedPreferences("session",
        Context.MODE_PRIVATE);

// escrever: apply() grava em segundo plano
prefs.edit()
    .putString("current_user_id", userId)
    .apply();

// ler: síncrono
String saved =
    prefs.getString("current_user_id", null);
```

```java
// DataStore: datastore-preferences-rxjava3
RxDataStore<Preferences> store =
    new RxPreferenceDataStoreBuilder(
        context, "session").build();
Preferences.Key<String> KEY =
    PreferencesKeys.stringKey("current_user_id");

store.updateDataAsync(prefs -> {
    MutablePreferences m =
        prefs.toMutablePreferences();
    m.set(KEY, userId);
    return Single.just(m);
});

Flowable<String> ids =
    store.data().map(p -> p.get(KEY));
```

FICHEIROS INTERNOS

# Anexos fora da base de dados

```
// a câmara pode terminar o processo da app:
// o caminho fica guardado antes de a abrir
fun newAttachmentFile(kind: AttachmentKind, filesDir: File): File {
    val dir = File(filesDir, ATTACHMENTS_DIR).apply { mkdirs() }
    // o nome do ficheiro é o id do anexo
    return File(dir, "${Uuid.random()}.${kind.extension}")
        .also { savedStateHandle[pendingKey(kind)] = it.absolutePath }
}

// a app da câmara escreve através de um content:// URI
takePicture.launch(FileProvider.getUriForFile(
    requireContext(), "${requireContext().packageName}.files", file))
```

### A Room guarda o caminho

Os bytes ficam em files/attachments/; a linha só guarda o path.

### Escrever, depois renomear

Um download vai para um .part e só passa a .jpg com renameTo().

### Apagar na ordem certa

Primeiro as linhas, numa transação; depois os ficheiros.

Fonte: ui/viewmodels/NewIncidentViewModel.kt · ui/fragments/NewIncidentFragment.kt · data/Repository.kt

SQLITE E ROOM

# Do SQLite à Room

```java
// SQLite direto (Java): SQL em texto,
// mapeamento à mão, erros só em execução
Cursor c = db.rawQuery(
    "SELECT serial_number, model FROM machine " +
    "ORDER BY serial_number", null);
List<MachineRow> rows = new ArrayList<>();
while (c.moveToNext()) {
    rows.add(new MachineRow(
        c.getString(0), c.getString(1)));
}
c.close();
```

```kotlin
// Room (Kotlin): SQL verificado ao compilar,
// mapeamento gerado pelo KSP
@Dao
interface MachineDao {
    @Query("SELECT * FROM machine " +
        "ORDER BY serial_number")
    fun observeAll(): Flow<List<Machine>>
}

// o Flow volta a emitir a cada alteração
machineDao.observeAll().collect { show(it) }
```

- **SQL verificado ao compilar**
- **Mapeamento gerado**
- **Consultas observáveis**
- **Migrações com esquema exportado**

ROOM

# Entidades

```
@Entity(
    tableName = "incident",
    foreignKeys = [ForeignKey(entity = Machine::class,
        parentColumns = ["id"], childColumns = ["machine_id"])],
    indices = [Index("machine_id")],
)
data class Incident(
    @PrimaryKey val id: Uuid,
    @ColumnInfo(name = "machine_id") val machineId: Uuid,
    val description: String,
    val severity: IncidentSeverity,
    @ColumnInfo(name = "reported_at") val reportedAt: Instant,
    @ColumnInfo(name = "sync_status")
    val syncStatus: SyncStatus = SyncStatus.Pending,  // por enviar
)
```

### id: UUID gerado na app

Cria-se sem servidor e é o mesmo antes e depois do envio.

### Chave estrangeira e índice

Não há ocorrência sem máquina; o índice acelera as consultas por máquina.

### Conversores de tipos

Uuid → BLOB, Instant → INTEGER (ms), enum → TEXT com o nome.

Fonte: data/model/Incident.kt (excerto) · data/UuidConverter.kt · data/InstantConverter.kt

ROOM

# DAO e operações CRUD

| Operação | Anotação | No FixTag |
|---|---|---|
| Criar | @Insert, @Upsert | upsert(incident) |
| Ler | @Query, com suspend ou Flow | observeAllWithStats() |
| Atualizar | @Update, @Upsert | update(attachment) |
| Apagar | @Delete | delete(incident) |

Relações: IncidentAndMachine junta cada ocorrência à sua máquina com @Embedded e @Relation, lidas numa só @Transaction.

```
// MachineDao (simplificado)
@Query("""
    SELECT machine.*,
        (SELECT COUNT(*) FROM incident
         WHERE incident.machine_id = machine.id
           AND incident.status != 'Resolved')
        AS open_incidents
    FROM machine
    ORDER BY machine.serial_number
""")
fun observeAllWithStats():
    Flow<List<MachineWithStats>>
```

O Flow volta a emitir sempre que machine ou incident mudam: o selo «1 em aberto» atualiza-se sozinho.

Fonte: data/model/Machine.kt (MachineDao, MachineWithStats) · data/model/Incident.kt (IncidentAndMachine)

ROOM · JAVA

# Room em Java

```java
@Entity(tableName = "machine")
public class Machine {
    @PrimaryKey @NonNull public String id;
    @ColumnInfo(name = "serial_number")
    @NonNull public String serialNumber;
    @NonNull public String model;
}

@Dao
public interface MachineDao {
    @Upsert void upsert(Machine machine);
    @Query("SELECT * FROM machine")
    LiveData<List<Machine>> observeAll();
}

executor.execute(() -> dao.upsert(m)); // fora da UI
```

## DE KOTLIN PARA JAVA

| Kotlin | Java |
|---|---|
| suspend | Executor ou ListenableFuture (room-guava) |
| Flow | LiveData ou Flowable (room-rxjava3) |
| withTransaction { } | runInTransaction(…) |
| ksp | annotationProcessor |
| CoroutineWorker | Worker |
| DataStore com Flow | RxDataStore |
| kotlin.uuid.Uuid | java.util.UUID |

OFFLINE-FIRST

# Repository

```
suspend fun createIncident(incident: Incident,
                           attachments: List<Attachment>) {
    // tudo ou nada: a ocorrência e os seus anexos
    db.withTransaction {
        db.getIncidentDao().upsert(incident)
        db.getAttachmentDao().insertAll(attachments)
    }
    syncScheduler.syncNow()   // agenda o envio (desligado na demo)
}

fun observeIncidents(): Flow<List<IncidentAndMachine>> =
    db.getIncidentDao().observeAllWithMachine()
```

### Escrever: Room primeiro

A ocorrência existe assim que é gravada; o envio vem depois.

### Ler: observar a Room

Flow → StateFlow no ViewModel → Fragment com repeatOnLifecycle.

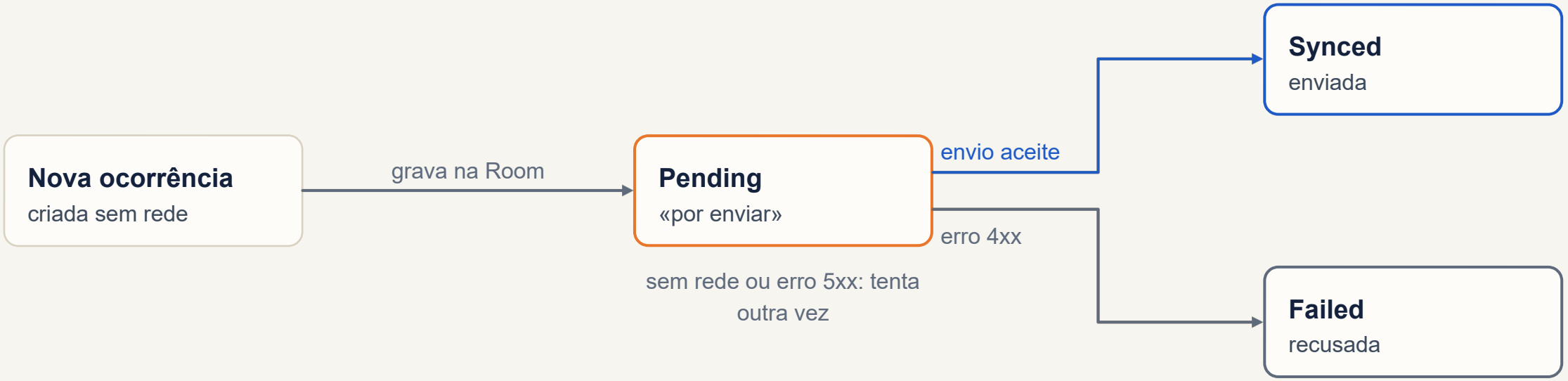
### Uma só porta

Os ViewModels só falam com o Repository, nunca com a API nem com os DAO.

Fonte: data/Repository.kt · ui/viewmodels/IncidentsViewModel.kt

OFFLINE-FIRST

# Estado de sincronização




### Reenviar é seguro

O id é gerado na app: o envio é um PUT /incidents/{id}, que se pode repetir sem duplicar.

### «N por enviar» é uma consulta

SELECT COUNT(*) … WHERE sync_status = 'Pending', observada com Flow.

### Envio com WorkManager

Só corre com rede e volta a tentar com backoff. Na demo está desligado.

ROOM

# Migrações

```kotlin
// v2 acrescenta incident.assigned_to (Kotlin)
@Database(
    version = 2,
    entities = [/* … */],
    exportSchema = true,
    autoMigrations = [
        AutoMigration(from = 1, to = 2),
    ],
)
abstract class Database : RoomDatabase()
```

```java
// a mesma alteração, escrita à mão (Java)
static final Migration MIGRATION_1_2 =
        new Migration(1, 2) {
    @Override
    public void migrate(
            @NonNull SupportSQLiteDatabase db) {
        db.execSQL("ALTER TABLE incident "
                + "ADD COLUMN assigned_to TEXT");
    }
};
```

### `fallbackToDestructiveMigration()`

Se faltar uma migração, a Room apaga e recria as tabelas — incluindo o que ainda não foi enviado. Só serve durante o desenvolvimento.

### schemas/…/1.json no Git

A Room exporta o esquema de cada versão. É a partir dele que gera as AutoMigrations e que as testa com MigrationTestHelper.

DEMONSTRAÇÃO

# FixTag sem rede

1. Modo de voo ligado; variante demoDebug instalada
1. Entrar como Operador: o perfil fica no DataStore
1. Máquinas: lista lida da Room, com o estado de cada uma
1. Nova ocorrência com foto e áudio: «3 por enviar»
1. Forçar a paragem da app e voltar a abrir
1. Sessão, ocorrência e anexos continuam lá
1. Database Inspector: a linha com sync_status = 'Pending'
1. Apagar a ocorrência pendente: saem as linhas e os ficheiros

## ONDE VER

Device Explorer, em /data/data/com.example.fixtag.demo/

```
files/datastore/
  session.preferences_pb
files/attachments/
databases/database
```

App Inspection → Database Inspector

```
SELECT COUNT(*) FROM incident
WHERE sync_status = 'Pending'
```

BALANÇO

# Vantagens e limitações

## Vantagens

- Registar uma avaria não depende do servidor
- A interface responde logo: lê sempre do disco
- Menos tráfego: só se envia o que está pendente
- Reenvios seguros, graças ao UUID gerado na app

## Limitações

- Conflitos quando duas pessoas mudam o mesmo registo
- Consistência eventual: o ecrã pode estar desatualizado
- Mais estados para gerir: pendente, enviado, falhado
- Dados no dispositivo: espaço, segurança e cópias de segurança
- Cada mudança de esquema obriga a uma migração

BALANÇO

# Erros frequentes

| Erro | O que acontece | Como evitar |
|---|---|---|
| Ler ou escrever na main thread | A Room lança uma exceção; as SharedPreferences podem causar ANR | suspend e Flow (em Java, um Executor) |
| Dois DataStore para o mesmo ficheiro | Exceção em tempo de execução | Um só, declarado no topo de um ficheiro |
| fallbackToDestructiveMigration em produção | Apaga o que ainda não foi enviado | Migrações e esquema exportado no Git |
| Ids atribuídos pelo servidor | Nada se cria sem rede; um reenvio duplica o registo | UUID gerado na app |
| Ficheiro sem linha na Room, ou o inverso | Espaço perdido ou anexos partidos | Transação primeiro, ficheiros depois |
| Auto Backup sem regras | Base de dados e tokens copiados para a nuvem | Excluir databases/ e files/datastore/ das cópias |

PARA A TURMA

# Questões para discussão

1 Duas pessoas alteram a mesma ocorrência sem rede. Quando a rede volta, qual das versões deve ficar — e quem decide?

2 Uma atualização muda o esquema da base de dados e há registos por enviar. Migrar, apagar e recomeçar, ou obrigar a sincronizar antes?

RECURSOS

# Código e protocolo

| | |
|---|---|
| REPOSITÓRIO | github.com/0xmaiev/FixTag , branch `entrega-10-27` |
| PROTOCOLO | `docs/protocolo-experimental.pdf` |
| VARIANTE | demoDebug, em Build Variants no Android Studio |
| SEM SEGREDOS | A variante demo não tem servidor nem login: os dados de exemplo são criados no primeiro arranque. |